



EE HPC - A quickstart guide

This document provides a guide on using the EE HPC GPU cluster for Stellenbosch University, particularly for training machine learning models using python.

Reasonable use

- *Only use the GPU cluster if your job requires substantial GPU usage.* If your massive job needs powerful CPUs or other specs, use the regular HPC machines and follow the [guide on the main Stellenbosch HPC page](#)
- Do not spam GPU nodes with jobs. Many people need to make use of the cluster, so **do not use up all the GPU nodes**.

Having a couple of jobs is fine, but if you are launching 4+ jobs and using up most of the GPU cluster for any substantial amount of time, then it is not fair on others who need to use it. Your jobs (and training progress) might magically dissapear in this case as well.

- Adhere to the general [Stellenbosch HPCs Acceptable Use policy as well](#).

Hardware layout

TL;DR the GPU cluster contains **8 nodes**. The node name and hardware for each node is as follows:

Node name	CPU specs	GPU specs	Local SSD space	RAM
comp047	2x Intel Xeon Gold 5218	3x Tesla T4 (16GB each)	62TB*	376GB
comp048	2x Intel Xeon Gold 5218	3x Quadro RTX 4000 (8GB each)	1.8TB	376GB
comp049	2x Intel Xeon Gold 5218	3x Quadro RTX 4000 (8GB each)	1.8TB	376GB
comp050 BROKEN	2x Intel Xeon Gold 5218	3x Quadro RTX 4000 (8GB each)	1.8TB	376GB

Node name	CPU specs	GPU specs	Local SSD space	RAM
comp051	2x Intel Xeon Gold 5218	3x Quadro RTX 4000 (8GB each)	1.8TB	376GB
comp054 BROKEN	2x Intel Xeon Gold 5218	3x Quadro RTX 4000 (8GB each)	1.8TB	376GB
comp055	2x Intel Xeon Gold 5218	3x Quadro RTX 6000 (23GB each)	1.8TB	376GB
comp056	2x Intel Xeon Gold 5218	3x Quadro RTX 6000 (23GB each)	1.8TB	376GB

Notes:

- Each node has 2x Intel(R) Xeon(R) Gold 5218 CPU with 16 cores each, totalling 32 logical processors.
- The 1.8TB SSD storage is under `/scratch-small-local` on each machine. Use it as a temporary data directory during your jobs and remove your data once done.
- * `comp047` has 62TB in `/datahome/scratch-small-local` and no regular `/scratch-small-local`, unlike the other nodes.
- More SSD storage under `/scratch-large-network`, but IO much slower than `/scratch-small-local`, so if your data can fit in the local scratch, rather do that.

Prerequisites

- Passing familiarity with Stellenbosch's HPC job and login system. See [here](#) for details.
- Have a terminal capable of running common bash commands (ssh, scp, rsync).
- Have access to the EE HPC GPU cluster.
- Be able to connect to the HPC1 server on the Stellenbosch network. i.e. `ssh <username>@hpc1.sun.ac.za` must not timeout.

Very quick start

1. Login to hpc1.sun.ac.za

2. Run: `qsub -I -l walltime=2:00:00 -q ee -l select=1:ncpus=4:mem=4GB:ngpus=1:Qlist=ee`

If you want to use python straight away, you can run `module load python/3.8.1` in the terminal and then `python3` should have several of the basic packages installed.

3. (when finished), type `exit` in the terminal to quit the job.

The 2nd command will launch you into an interactive session on the EE HPC server.

Details

Taking a look at the `qsub` command again:

- `-I` : launch an *interactive* instance. Usually the instance is launched from a special job script as specified on the HPC website; using this command makes the job launch from the terminal instead. Typically for normal jobs we leave this out.
- `-l` : a resource list of the format `-l value1=key1,value2=key2,...`
- `walltime` : how long the job will remain active for, in the example about this is 2 hours.
- `-q ee` : this sets the destination for the job as being on the EE GPU server, instead of the generic HPC1 servers.
- `-l select=1:ncpus=4:mem=4GB:ngpus=1:Qlist=ee` : selects the number of CPUs, GPUs, and amount of RAM for the job. You can also specify a `host` argument to select the specific GPU node you would like to use. E.g. using `-l select=1:ncpus=4:mem=4GB:ngpus=1:Qlist=ee:host=comp047` would ensure that your job launches to the `comp047` node with the hardware as specified in the table at the top of this document.

Each of these arguments can also be specified inside a job script -- please see the details on the [HPC website for more info](#).

Setting up your first job

1. Push your data to your HPC home drive

Typically this only needs to be done once. This also assumes your dataset is less than 1TB. To do this we use `rsync` -- simply run:

```
>> rsync -vax --progress <source directory of dataset> <username>@hpc1.sun.ac.za:~/dataset
```

Also make sure your source code and

2. Setting up the bash script

Here you have the most freedom, but here is a decent recipe for Python jobs with custom conda dependencies:

A note about this recipe:

- It assumes you have an `environment.yml` conda environment file in your HPC home directory. This is the environment necessary to run your script.
- It assumes your code is in the root directory of your HPC folder, with a `train.py` script.

```
#!/bin/bash
#PBS -N JobName
#PBS -l select=1:ncpus=4:mem=16GB:ngpus=1:Qlist=ee
#PBS -l walltime=6:00:00
#PBS -m ae
```

```

#PBS -e output.err
#PBS -o output.out
#PBS -M <username>@sun.ac.za

# make sure I'm the only one that can read my output
umask 0077
# create a temporary directory with the job ID as name in /scratch-small-local
SPACED="{PBS_JOBID//.-}"
TMP=/scratch-small-local/${SPACED} # E.g. 249926.hpc1.hpc
mkdir -p ${TMP}
echo "Temporary work dir: ${TMP}"

cd ${TMP}

wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -O miniconda.sh
bash miniconda.sh -b -p miniconda

# Ensure miniconda is activated
echo "Activating conda"
source ./miniconda/bin/activate

# copy the input files to ${TMP}
echo "Copying from ${PBS_O_WORKDIR}/ to ${TMP}/"
/usr/bin/rsync -vax "${PBS_O_WORKDIR}/" ${TMP}/

# write my output to my new temporary work directory
conda env create -f environment.yml
export LD_LIBRARY_PATH=${LD_LIBRARY_PATH}:${TMP}/miniconda/lib/
conda activate <env name>
# You may need to add additional lines here if your script
# requires custom git / pip dependancies not included in your conda env.
python train.py <args>

# job done, copy everything back
echo "Copying from ${TMP}/ to ${PBS_O_WORKDIR}/"
/usr/bin/rsync -vax ${TMP}/ "${PBS_O_WORKDIR}/"

# if the copy back succeeded, delete my temporary files
cd ..
[ $? -eq 0 ] && /bin/rm -rf ${TMP}

```

It should be fairly simple to adapt this bash script to work for your case and your dependancies. I recommend not using more than 3 GPUs at once unless you feel comfortable with cross-node communication and multi-gpu support. Rather focus on optimizing inter-GPU communication on the 3 GPUs on the single node (physical machine) before attempting multi-node communication.

Notes:

- You not need to use conda/miniconda if you do not need it for your dependancies. You can also use the built-in python module (you can activate it once in a job with `module load python/3.8.1`), however it will not work for any non-standard dependancies.
- Like with the interactive script, you can ensure your job runs on a desired node by adding the `host` argument. E.g by changing the line

```
#PBS -l select=1:ncpus=4:mem=16GB:ngpus=1:Qlist=ee to
```

`#PBS -l select=1:ncpus=4:mem=16GB:ngpus=1:Qlist=ee:host=comp047` , you will ensure that your job runs on node 47 with the Tesla T4 GPUs.

3. Run the script

Now place your bash script in your HPC home directory (the directory you are in when you log into `hpc1.sun.ac.za`). You can again use `rsync` for this.

Now, to start the job, simply run: `qsub myscript.sh`

You can check up on how a job is currently doing with the [commands available on the HPC website](#), namely you can peek at the terminal output of your bash script with `qpeek <job ID> -t`

Debugging

If your job fails or does not run for some reason, it may be useful to enter an interactive session and try run your bash script on a small instance and run the commands line-by-line to see where the problem is. Your job's output and error messages will also be in the `output.out` and `output.err` command as indicated by the first few lines of the starter bash script:

```
#PBS -e output.err
#PBS -o output.out
```

Advanced topics

Caching conda packages in home directory

If cloning your conda environment is taking too long each run (because it has to download the packages from the internet again), you can store caches in your home directory. To do this:

1. Create your desired environment in an interactive session (using `miniconda` commands similar to the script above) with all the packages you need
2. Copy the `miniconda` cache folder to your home drive: `rsync -vax <path to miniconda install>/pkgs ~/.conda/pkgs`

Done! Now, whenever your job tries to create an environment which shares packages with the one you just copied the cache of, it will grab the packages from your home directory instead of from the internet.

Final notes

If any arguments aren't clear or additional detail is needed, it is likely worth consulting the [Stellenbosch HPC website](#) -- it has all the additional tips and tricks.

Note: your HPC home directory is not on the same physical machine as the HPC machine, so IO operations between the GPUs and your HPC home directory will be slow. Rather use the method in the script above (copying your data to `/scratch-small-local`) for fast IO between the SSD storage and GPUs.

And finally, remember to follow the acceptable use and citation policy as specified [here](#).

Authors: Matthew Baas and Kevin Eloff
